G R O U P 3 D O C U T R U S T

DocuTrust Defence Handbook

Seventy-nine likely examination questions with model answers, per-student defence packs for all six members, and the eight core concepts explained for absolute beginners

79 questions across six themes: architecture, Java, security, database, hard challenges, mobile

6 per-student packs: owned modules, feature claims, pitch scripts, personal Q&A

Appendix: hooks, architecture, folders, database, server, tech stack and the data API - like I'm 5

Companion to the DocuTrust Project Guide and the repository docs

Table of Contents

How to Use This Handbook.....

Section A — Architecture and Design (Q1–Q12).....

Section B — Java Backend Without Spring (Q13–Q24).....

Section C — Authentication and Security (Q25–Q38).....

Section D — Database and Transactions (Q39–Q50).....

Section E — The Hardest Challenges, Deep Dive (Q51–Q62).....

Section F — React Native, MVP, Testing and Teamwork (Q63–Q78).....

Per-Student Defence Packs.....

- Student 1 — Infrastructure & Database.....
- Student 2 — Auth & OTP Security.....
- Student 3 — Business Logic (Documents, Sharing, Verification).....
- Student 4 — Mobile App (React Native 0.88 / Expo SDK 58).....
- Student 5 — The 8-Pass Security Audit.....
- Student 6 — Deployment & DevOps.....

Appendix — The Eight Core Concepts, Explained for Absolute Beginners.....

1. React Hooks — the sticky notes that remember things (rehearse: Student 4).....
2. The Overall Architecture — the customer, the waiter, the kitchen (rehearse: Student 1).....
3. The Frontend Folders — it's like a smartphone (rehearse: Student 4).....
4. The Backend Folders — a well-organized restaurant (rehearse: Student 1).....
5. The Database — a highly organized filing cabinet (rehearse: Student 1).....
6. The Server — a 24/7 receptionist answering phones (rehearse: Student 6).....
7. The Tech Stack — what we chose and why (rehearse: Student 6).....
8. The Data Consumption API — the standardized order form (rehearse: Student 3).....

Note: This Table of Contents is generated via field codes. To ensure page number accuracy after editing, please right-click the TOC and select "Update Field."

How to Use This Handbook

This handbook prepares all six members of Group 3 for the individual oral defence. Part one contains **seventy-eight questions** grouped into six themes, from overall architecture down to the trickiest follow-ups about revocation and token reuse. Part two contains **six per-student defence packs**, each mapping one student to the modules they own, the numbered features they can claim, a two-minute pitch script, and five personal questions with model answers. The questions are the ones lecturers most commonly ask for this exact brief: a React Native and Expo client, a pure Java backend without Spring, PostgreSQL on Supabase, and the six hard challenges named in the project statement.

Rehearse in three passes. First read your own pack aloud twice, because your two-minute pitch is the only part of the defence you fully control. Second, work through all seventy-eight answers in pairs, one person asking and one answering without looking; anything you cannot answer in thirty seconds goes on a revision card. Third, run the demo script while answering — a demonstration attached to an answer is worth more than the answer alone. Keep the star habit: state the claim, show the mechanism, then name the evidence in the repository (a file, a test, or an edge-case script) that proves it.

Three universal answers are worth memorising verbatim. When asked why no framework: the brief forbids Spring and JUnit, and writing the router, middleware, JSON layer and test runner ourselves turned hidden framework decisions into examinable code. When asked what you are proudest of: the security chain — encrypted at rest, streamed through guarded endpoints, guarded by one atomic SQL UPDATE, audited by append-only triggers — because every claim is demonstrable live. When asked what you would improve: background notification delivery outside the app (push), object storage with server-side encryption for the vault, and moving integration tests into a containerised pipeline.

Theme	Questions	Typical examiner angle
A. Architecture and design	Q1–Q12	Big-picture choices and their justifications
B. Java backend without Spring	Q13–Q24	How the plumbing actually works
C. Authentication and security	Q25–Q38	JWT, hashing, encryption, MFA, rate limiting
D. Database and transactions	Q39–Q50	Schema, constraints, migrations, concurrency
E. Hardest challenges deep dive	Q51–Q62	The six named challenges, edge cases, attacks
F. React Native, MVP, tests, teamwork	Q63–Q78	Client patterns, testing strategy, roadmap defence

Table 1. Question bank map.

Section A — Architecture and Design (Q1–Q12)

Q1. Walk us through the system in one minute.

DocuTrust is a document vault with three parts. A React Native Expo client with 45 screens talks over HTTPS to a pure-Java REST server; the server keeps metadata in PostgreSQL on Supabase and file bytes encrypted with AES-256-GCM in a server-side vault. Everything a user does — upload, share, verify, review — goes through one router, one middleware chain and one set of services, and every security-relevant action is written to an append-only audit trail. The three roles are member, org_manager and admin, all enforced server-side.

Q2. Why a modular monolith instead of microservices on the backend?

The brief says microservices in the frontend, not the backend, and for a six-person, three-week project a modular monolith is the honest engineering choice: one deployment, real transactions across modules, no network calls between services to fail. We kept the benefits microservices give — independent modules, clear boundaries, small readable files — by enforcing a strict Router, Controller, Service, DAO layering with no skipping. If the system grew, each service package already has a single reason to exist, so extracting one into a process later would be a packaging change, not a redesign.

Q3. Why did you write your own JSON parser — is that not risky?

It was a deliberate trade-off forced by the no-heavy-framework brief, and we treat it as a teaching asset rather than a liability. The parser is a recursive-descent implementation of the JSON grammar — objects, arrays, strings with escapes, numbers, booleans and null — covered by unit tests for round-trips and malformed input. In production we would use a hardened library such as Jackson, and we say exactly that in the defence: the point of this project was to demystify what libraries do, not to claim a student parser belongs in a real deployment.

Q4. What is the request lifecycle from tap to response?

A tap triggers a presenter, which calls the ApiClient, which sends the request with the stored Bearer token. On the server the router matches the path, then middleware runs in order: CORS, logging, JWT authentication, role guard, rate limiting. The controller parses the request into plain data and calls a service; the service enforces business rules inside a transaction using DAOs with PreparedStatements; the response goes back as a JSON envelope with ok, data and optional meta. Errors bubble up as typed exceptions that one global handler converts into the error envelope with the right status code.

Q5. Why is ownership checked in SQL rather than in Java code?

Because a WHERE clause cannot be forgotten the way an if-statement can. Every document query is scoped with AND owner_id = ?, so the database itself refuses to return rows the caller does not own, and a request for someone else's document yields a 404 rather than a 403 — the server does not even confirm the row exists. This is defence in depth: even if a future code path skipped a service-level check, the SQL shape still protects the data. The edge-case script proves it with two accounts.

Q6. Explain the three roles and what each can do.

Member is the default: a personal vault with documents, folders, sharing, verification and notifications. Org_manager additionally manages their organisation's members — invites, role changes, removals — through guarded endpoints. Admin works the platform: the review queue for sensitive documents, the user directory with suspension, a read-only platform-wide document view, and the audit log browser. Role checks are one-line guards per route plus audit rows on every refusal, and the admin surface in the app is hidden by role but never trusted — the server is what refuses.

Q7. What is the admin review process and when does it trigger?

Uploads of type medical or financial cannot become active immediately: a schema CHECK forbids a sensitive document to be active unless its review_status is submitted, so the service parks the upload as pending_review and opens an admin_reviews row. An admin works the queue — joined against its target so the card shows full context — and approves, rejects with a note, or escalates; every decision updates the target, notifies the owner and writes an audit row. The same queue also handles suspicious user accounts and flagged verification requests, which is why it is a process, not a screen.

Q8. How does the frontend know what the backend offers?

Through a written contract, docs/API_CONTRACTS.md, that both sides coded against before either was finished: paths, methods, guards, request and response shapes, the envelope, the error codes and the pagination convention. On the client the contract is re-expressed once in endpoints.js, so no screen hardcodes a URL; on the server the route table mirrors it. The /api/meta endpoint also exposes machine-readable metadata — name, tagline, roles, screen and feature counts — that the About screen renders.

Q9. Why PostgreSQL and not MongoDB?

DocuTrust's data is relational and rule-heavy: shares belong to documents, versions belong to documents, and the rules — one target per share, sensitive documents must be reviewed, version numbers unique per document — are exactly what foreign keys, CHECK constraints

and UNIQUE indexes express best. MongoDB would push those invariants into application code, which can have bugs; Postgres enforces them on every statement, including manual ones. We also rely on Postgres features a document store lacks: real transactions with isolation for the atomic share guard, triggers for append-only audit tables, and partial plus trigram indexes for our query patterns. Supabase simply hosts that same engine.

Q10. What is MVP and why did you choose it over MVVM or MVC?

MVP is Model-View-Presenter: the view is a dumb React component, the model is the data layer, and the presenter holds the logic and a small state machine, with the view never touching the model directly. We chose it because presenters are plain classes, so they unit-test in Jest with mocked services and no React rendering at all, which the brief's automated-tests requirement rewards. MVVM's two-way bindings are a poor fit with React's one-way data flow, and MVC tends to let fat controllers accrete view knowledge. MVP also gives the whole app one consistent lifecycle — idle, loading, success, error, empty — so every screen behaves identically.

Q11. What are frontend microservices in this project, concretely?

Nine self-contained modules — auth, documents, folders, sharing, verification, notifications, admin, org, profile — each owning its API calls, models and presenters behind hard walls. Screens are dumb views registered in a central SCREEN_REGISTRY that navigation, deep links and notification taps all resolve through, so no module imports another module's internals. The payoff is ownership and replaceability: the sharing module can be rewritten without touching documents, and where each piece of logic lives has a one-directory answer. It mirrors the backend's modular monolith deliberately — same walls, same story.

Q12. How does the SEO metadata requirement fit a mobile app?

Search engines cannot crawl a native app, so the SEO surface is the one public web page the system serves: the verification link. When a browser asks for a verification token, the Java service renders a lightweight HTML page with title and meta description, Open Graph tags for social previews and JSON-LD structured data for the verification event — content-negotiated from the same endpoint that returns JSON to the app. The Expo config additionally carries the app's name, description and deep-link scheme, and /api/meta exposes structured metadata for integrations. So the requirement is met where SEO is actually meaningful.

Section B — Java Backend Without Spring (Q13–Q24)

Q13. How does your server work without Spring?

It is a plain Java program using com.sun.net.httpserver.HttpServer, the JDK's built-in HTTP engine — the same one application servers use underneath. Main.java creates the server on port 8080, registers 71 route paths on a hand-written router, applies the middleware chain, runs migrations, seeds demo data and starts a scheduler. Spring's job — dependency wiring, routing, filters, JSON — is done by about two thousand lines of our own small, explicit code. Every mechanism the framework would have hidden is now a file we can open and defend.

Q14. Explain your router. How does /api/documents/:id match?

Routes are registered as method plus pattern plus handler. At startup the router splits each pattern into segments and stores them; at request time it splits the incoming path the same way and compares segment by segment — literal segments must be equal, colon-prefixed segments match anything and capture their value into a map. Exact static segments win over parameters when both match, and method mismatch yields 404/405 handling. Query strings are parsed into a map on the same context object, so a handler reads ctx.pathParam(id) and ctx.query(search) without touching raw exchange internals.

Q15. What is middleware in your design, and what does each piece do?

Middleware is a wrapper applied around a handler before any business code runs. Our chain is CORS (answer preflights, stamp allowed origins), logging (one line per request with status and duration), authentication (verify the JWT, load claims, produce precise 401s), role guard (compare the role claim, refuse with 403 and audit), and rate limiting (sliding window per IP for auth and public endpoints). Handlers are composed functionally — requireRole(admin, authenticate(h)) — so the chain reads like the security policy it enforces. Because it is our own code, we can show exactly where each check happens and in what order.

Q16. How do you handle multipart file uploads without libraries?

The context object reads the raw body bytes, extracts the boundary from the Content-Type header and splits the parts, parsing each part's headers for name and filename and returning a map of fields plus the file part's bytes and metadata. Upload size is capped before buffering to prevent memory abuse, and the allowed mime types are checked immediately. The file then goes straight into streaming SHA-256 computation and AES-GCM encryption, so plaintext never lingers. The e2e test exercises this with a hand-built multipart body, which doubles as proof the parser is correct.

Q17. How does your connection pool work?

It is a small thread-safe pool backed by a blocking queue: on start it opens a configurable number of PostgreSQL connections and validates them; getConnection() borrows one or blocks briefly, and close() returns it rather than closing the socket. Stale connections are detected by a lightweight validity check and replaced on borrow. DAOs always use try-with-

resources so a connection is returned even when a statement throws, and transactions borrow one connection and keep it for the whole unit of work. In production this would be HikariCP; the point here was to show we understand what a pool does and why N connections bound throughput.

Q18. Why PreparedStatements, and what do they prevent?

PreparedStatements send SQL structure and data separately: parameters travel as typed values, never spliced into the SQL text, which makes SQL injection structurally impossible in our DAO layer — there is no string concatenation of user input anywhere. They also let the driver cache execution plans, and they document intent: every query's shape is visible at the top of the DAO method. The defence demonstration is simple: search for a title containing a quote and an OR 1=1 attempt; the trigram search returns rows whose title literally contains that text, and nothing else happens.

Q19. What does your migration runner do?

On boot, before the server accepts traffic, the runner reads the numbered V01–V11 SQL files, compares them against a schema_migrations table recording what has already been applied, and executes the new ones in order inside a transaction each. Applied files are never re-run, so restarts are idempotent, and a failure mid-file rolls the whole file back rather than leaving half a schema. The team applies the same files by hand in Supabase's SQL editor when demonstrating, which shows the runner is a convenience, not a lock-in. This is exactly how Flyway works, minus the dependency.

Q20. How are errors handled across the whole server?

Business code throws a single typed AppException carrying an HTTP status, a machine-readable code and an optional field. One global boundary catches exceptions, converts known types into the error envelope { ok:false, error:{code,message,field} } and unknowns into a 500 INTERNAL_ERROR with the details logged server-side and an audit row — never leaked to the client. Middleware-generated failures such as 401, 403 and 429 use the same envelope, so the client's error handling has exactly one shape to parse. The client maps codes to behaviour: TOKEN_EXPIRED refreshes and retries, TOKEN_REUSE logs out, VALIDATION_ERROR shows field errors.

Q21. Is your server multi-threaded? What does that imply?

Yes — HttpServer dispatches each exchange on a worker thread pool, so handler code must be thread-safe. We keep handlers stateless: anything mutable lives either in the database, in the per-request context object, or in deliberately concurrent structures such as the rate limiter's window and the download-ticket map, which use concurrent collections and atomic operations. JDBC connections are never shared between threads — each request borrows its

own from the pool — and transactions never escape the thread that opened them. The scheduler runs on its own single-threaded executor so its database work cannot interleave badly with request work.

Q22. How would you scale this server?

Three honest stages. Vertically first: the server is stateless apart from the vault and the in-memory ticket map, so bigger machines and a tuned pool go a long way. Second, move the two pieces of in-memory state out — tickets into a short-TTL table or Redis, the rate limiter into a shared store — and then multiple instances can sit behind a load balancer with the vault on object storage. Third, split by load: the document download path and the API path scale differently, so extracting a dedicated file-streaming service with its own pool is the natural first microservice. The database read replicas and pg_trgm indexes cover the read-heavy search growth.

Q23. What is in your pom.xml and why?

The pom declares Java release 21 for the compiler plugin, exactly one runtime dependency — the org.postgresql JDBC driver — and the exec and jar plugins so the project runs both as mvn exec:java and as a fat-jar java -jar with the driver on the classpath. There is deliberately no test framework dependency: tests use our own DocuTest runner and run with a plain java command, because the brief forbids JUnit and adding TestNG would only re-import what we were told to avoid. Surefire is absent for the same reason; run-tests.sh drives the suite instead.

Q24. What happens on first boot with an empty database?

Migrations run first: V01 through V08 create extensions, tables, constraints, indexes, triggers and views, recorded in schema_migrations. Then the Bootstrap seeder checks whether the users table is empty and, if seeding is enabled, creates the demo organisation, six accounts with proper PBKDF2 hashes, folders, five documents in different lifecycle states, an active share and an approved verification request — pushing a generated PDF through the real encryption path so demo bytes are representative. Finally the scheduler starts its sixty-second tick and the health endpoint begins reporting database true. The console logs each stage, so bring-up is observable.

Section C — Authentication and Security (Q25–Q38)

Q25. What exactly is inside your JWT?

Three Base64-URL segments joined by dots. The header names the algorithm HS256 and type JWT. The payload carries the subject (the user's id), the role claim, the organisation id, the issued-at time, a fifteen-minute expiry and a unique token id. The third segment is an

HMAC-SHA256 signature over header-plus-payload made with a server secret loaded from the environment. We build and verify it with javax.crypto.Mac and compare signatures in constant time, so signature checking does not leak timing information.

Q26. Why does the access token live only fifteen minutes?

A short access token bounds the damage of a leak: a stolen bearer token is useful for at most fifteen minutes, and it cannot be revoked faster than its natural death because it is stateless. The refresh token is the long-lived, revocable half of the pair, and it only ever travels to one endpoint, so the window in which it can be intercepted is small. This asymmetry — short and stateless, long and stateful — is the standard trade-off, and the client hides it completely behind the silent refresh, so users never see a login prompt.

Q27. Walk through refresh rotation step by step.

The client posts its refresh token to /api/auth/refresh. The server hashes the presented token with SHA-256 and looks the hash up in refresh_tokens; the row must exist, be unrevoked and unexpired. If valid, the server revokes that row, records the id of its replacement in replaced_by, inserts a new row with a fresh random token, and returns a new access plus refresh pair. If the presented token's hash matches a row that is already revoked — meaning it was rotated before — the server answers 409 TOKEN_REUSE and revokes every token belonging to that user, ending the whole family.

Q28. Why is reuse detection important? What attack does it stop?

It stops the classic stolen-refresh-token attack. Suppose an attacker copies a refresh token; the real client refreshes first and the family rotates. When the attacker later replays the stolen token, the server sees an already-rotated token — something only a copy could present, because the legitimate client replaced its own — and knows the family is compromised. Revoking the whole family forces the thief to re-authenticate and alerts the real user, who must log in again. Without reuse detection, rotation is mere theatre; with it, rotation becomes an intrusion detector.

Q29. Why PBKDF2 and not bcrypt or Argon2? Why 120,000 iterations?

PBKDF2 is specified in the JDK standard library (PBKDF2WithHmacSHA256), and the brief restricts us to dependencies the JDK provides, so it is the correct choice inside those constraints; bcrypt and Argon2 are both excellent but require libraries we were not permitted. We use 120,000 iterations with a per-user sixteen-byte random salt and a 256-bit derived key, stored in the self-describing format pbkdf2$iter$salt$hash, which lets the iteration count be raised later without breaking old hashes. Verification is constant-time to avoid timing leaks, and the unit tests cover verify-fail and unique-salt properties.

Q30. Explain AES-256-GCM and why you chose that mode.

GCM is an authenticated encryption mode: it encrypts with AES using a 256-bit key and a fresh 12-byte initialisation vector per file, and it appends an authentication tag computed over the ciphertext. The tag means any later modification of the stored bytes — corruption or a deliberate tamper — is detected at decrypt time, which ordinary modes like CBC would not notice. Fresh IVs per encryption are mandatory in GCM; reuse would be catastrophic, so the vault generates one with SecureRandom for every write. The key comes from the environment, never from source control, and the unit tests prove tampering causes decryption to fail.

Q31. Where is the encryption key kept, and what happens if it is lost?

The master key is loaded from the VAULT_KEY environment variable as Base64; if it is absent in development the server generates an ephemeral key and prints a loud warning that previously stored files will not decrypt after a restart. Losing the master key in production means the vault is unrecoverable by design — that is the trade encryption makes — so operations would rotate to a backup key source such as a managed KMS. In the demo we show the warning, which itself is evidence the team thought about key lifecycle.

Q32. How is TOTP multi-factor authentication implemented?

RFC 6238, by hand. Setup generates a twenty-byte secret, Base32-encodes it, stores it on the user row and returns an otpauth:// URL that any authenticator app can scan. The server computes the six-digit code from the shared secret and the current thirty-second time step, comparing with a plus-or-minus one step window to tolerate clock drift, in constant time. Login for an MFA-enabled user returns a distinct 440 MFA_REQUIRED status until a valid code accompanies the credentials, and the app routes to a dedicated six-box code screen. The post-audit overhaul added the email fallback: a user without their authenticator device re-submits the login with useEmailMfa set, and the server answers 440 MFA_EMAIL_CHALLENGE with a ten-minute challenge token and emails a six-digit code; POST /api/auth/verify-otp exchanges challenge plus code for the real session tokens. The unit tests generate codes at the current and adjacent steps and assert accept and reject behaviour.

Q33. How does rate limiting work and where is it applied?

Token buckets from Bucket4j, one per scope-and-IP pair and held in a Caffeine cache so idle entries are evicted. Budgets are per scope: twenty authentication attempts per minute, thirty for the public share endpoints, twenty for verification lookups, ten for everything else. A bucket that runs dry answers 429 RATE_LIMITED, and because a token bucket refills continuously it also smooths bursts instead of allowing a whole window's worth of requests

in one stampede. It is applied in middleware, so it protects the whole route before any handler runs. In a multi-instance deployment this state would move to a shared store such as Redis, which we state openly.

Q34. What does your server log, and what does it never log?

Security-relevant events land in audit_logs: logins and failures, downloads, share opens, revocations, admin decisions, refused privilege escalations and internal errors — each with actor, entity, IP, user agent and a JSON detail payload. Request logging middleware writes one operational line per request with method, path, status and duration. We never log passwords, raw tokens, vault contents or reset codes; tokens appear only as hashes, and error responses to clients never include stack traces or SQL. The append-only trigger on audit_logs means even the logs cannot be quietly rewritten.

Q35. How do you prevent SQL injection, XSS and CSRF?

SQL injection: every query is a PreparedStatement with bound parameters; no user input is ever concatenated into SQL, and the search demo shows an injection attempt returning harmless literal matches. XSS: the API returns JSON, never HTML built from user input, and the one HTML page we render — the public verification page — is a fixed template with server-computed values and no user-supplied markup; React also escapes by default. CSRF: the API is not cookie-authenticated — the client sends an explicit Bearer header that a cross-site form cannot forge — and the state-changing public endpoints are token-parameterised, which is the standard bearer-token defence.

Q36. How are passwords reset without leaking who has an account?

Forgot-password always answers 200 regardless of whether the email exists, so responses cannot be used to enumerate accounts. For real users the server writes a hashed, single-use code with a fifteen-minute expiry to password_resets and the app's reset flow consumes it: correct code plus a new password, used once, then dead. Codes are hashed at rest like passwords, so a database leak does not reveal usable reset codes, and the unit tests cover the single-use and expiry behaviour.

Q37. What happens when an admin suspends an account mid-session?

Suspension is a status on the user row, and the authentication middleware loads the current status with the token claims on every request, so a suspended user's very next call — even one with a perfectly valid, unexpired JWT — is refused with a 403. The refresh endpoint also refuses suspended accounts, so waiting for the access token to expire does not help. There is a guard preventing an admin from suspending themselves, and the whole action is written to the audit log with both actor and target.

Q38. Which OWASP-style risks did you consciously accept, and why?

Three, stated honestly. The vault key lives in an environment variable rather than a managed KMS — acceptable for the coursework scope, with KMS named as the production answer. Rate limiting state is in-memory, which is correct for one instance and documented as needing Redis when horizontally scaled. The third was email delivery being stubbed in-app — and that one is now fixed: the OTP overhaul wired a real transactional email provider (Brevo) behind a MAIL_MODE switch, with a dev mode that echoes codes in an X-Dev-OTP header for demos and a live mode that sends real email. The remaining honest gaps are the two above, plus push notifications being absent (the in-app inbox is the transport). Naming accepted risks with their mitigations is part of a good defence.

Section D — Database and Transactions (Q39–Q50)

Q39. Describe the schema in thirty seconds.

Thirteen tables in four groups. Identity: organisations, users, refresh_tokens, password_resets, settings. Vault core: folders, documents, document_versions, document_history. Sharing and proofs: shares, verification_requests. Platform: notifications, admin_reviews, audit_logs. Documents carry a six-state lifecycle and an expiry date; versions pin SHA-256 digests; history and audit tables are append-only by trigger; every child table cascades from its parent except where the business says otherwise, like folders detaching documents instead of destroying them.

Q40. Which constraints carry business rules, not just integrity?

Three stand out. A CHECK on shares requires a user target or a link token — a share must be shareable somehow. A CHECK on documents forbids a medical or financial document to be active unless its review_status is submitted, which hard-wires the administrative review process into the engine. And a UNIQUE on (document_id, version_number) turns a concurrent-upload race into a clean conflict instead of corrupt state. Roles, statuses, document types, permissions and reminder windows are all CHECK-enumerated too, so even a bug cannot write an impossible value.

Q41. What is a transaction, and where do you use them?

A transaction groups statements so they all commit together or all roll back — the database never observes half of one. Every multi-step write in DocuTrust runs in one: upload inserts the document, version one and history (and possibly a review row) atomically and deletes the staged file if anything fails; permanent delete removes rows and encrypted bytes together; a share open is a single guarded UPDATE whose row-count tells the whole story. The Tx

helper sets autocommit false, commits on success and rolls back on any exception, so services declare intent and never manage connection plumbing.

Q42. Two people open the last view of a share link at the same second. What happens?

Exactly one of them gets in. The guard is a single conditional UPDATE — increment view_count WHERE the share is unrevoked, unexpired and under its max_views — and Postgres row locks make those two updates serial, so the second one matches zero rows. The service then re-reads the row to answer precisely why the link is dead: expired, revoked or exhausted, returning 410 or 409 accordingly. No count can overshoot because the increment and the guard are the same statement.

Q43. Two uploads race to create version N+1. What happens?

Both read the current max, both compute N+1, but the UNIQUE constraint on (document_id, version_number) lets only the first insert commit. The loser receives a constraint violation, which DocumentService catches and rethrows as a 409 CONFLICT with a clear message, inside a rolled-back transaction — no orphan file, no phantom version. The client surfaces it as a friendly retry prompt. This is the canonical use of a database constraint as a concurrency primitive, and the edge-case script fires it from two terminals.

Q44. Explain the append-only triggers. Why not just enforce it in code?

BEFORE UPDATE OR DELETE triggers on document_history and audit_logs raise an exception unconditionally, so the tables physically cannot be rewritten — by the app, by a migration, or by an administrator with a SQL console. Code-level enforcement would be one forgotten WHERE clause away from silent history edits, and audit tables are exactly where you must assume someone will eventually try. The defence demo types an UPDATE and shows the database throwing append-only exceptions live. The same philosophy puts automatic history-writing into AFTER INSERT/UPDATE triggers on documents, so no code path can skip the trail.

Q45. How do your indexes match your queries?

Index choice follows the hot paths. Owner-scoped lists use a partial index on documents(owner_id) filtered to alive rows, because deleted documents are queried rarely but listed constantly. Search uses a GIN trigram index on titles so ILIKE patterns stay indexed instead of scanning. Token lookups — which happen on every public share hit — use a partial index on link_token filtered to unrevoked shares. History and audit are queried newest-first per entity, so they carry composite indexes on (entity, created_at DESC), and the notifications inbox indexes (user, read_at, created_at) to answer unread-first queries.

Q46. What are the two views for, and why SQL views?

v_expiring_documents computes the expiry radar — days left per document inside its reminder window — and v_admin_stats aggregates the dashboard counters in one select. Putting them in SQL means one tested definition the server queries directly, instead of the same date arithmetic scattered through Java or JavaScript where two implementations could drift. Views also express intent: the dashboard's meaning lives next to the data it summarises. They are ordinary views, so they are always current, which matters because the expiry scheduler rewrites statuses every sixty seconds.

Q47. Why is documents.sha256 duplicated from document_versions?

It is a deliberate denormalisation kept honest by a trigger: an AFTER INSERT trigger on document_versions copies the new version's digest, size and mime type onto the parent document row and bumps current_version. The hottest integrity check — comparing stored versus recomputed digest on download — then reads one row instead of joining, and list screens show integrity state without a join. Because the trigger maintains it, the mirror cannot drift: there is no code path that writes a version without updating its document's head.

Q48. How does soft delete work, and what is the danger with it?

Soft delete sets status to deleted and stamps deleted_at, so trash can restore a document and history stays meaningful. Every hot query filters deleted_at IS NULL — which is what the partial owner index is built around — and permanent delete from the trash screen is the only path that removes rows and encrypted bytes. The classic soft-delete danger is forgetting the filter somewhere and leaking deleted rows into lists, shares or search; our answer is that all list queries funnel through one DAO method family that applies the filter, and the share guard also refuses shares pointing at deleted documents server-side.

Q49. How does the scheduler update statuses without spamming users?

Every sixty seconds the ExpiryScheduler marks active documents whose expiry falls within their reminder window as expiring, and expiring documents past expiry as expired, then raises notifications. The de-duplication rule is one notification per document per type per day: before inserting, it checks whether today's notification already exists for that entity and type, so a document sitting in the expiring state for a fortnight produces fourteen reminders at most, not thousands. Expired verification requests are retired in the same tick. It runs on its own executor thread with its own connection so it never contends with request transactions.

Q50. Why store files on disk instead of in the database or on Supabase Storage?

Databases are poor filesystems: large bytea rows bloat backups, slow dumps and complicate streaming, while DocuTrust needs range streaming, checksum-verified reads and cheap

deletes. We chose a server-side encrypted directory as the storage engine behind a small FileVault interface, which keeps the demo fully self-contained and lets us demonstrate tamper detection physically. The interface is one seam: swapping in Supabase Storage or S3 with server-side encryption means implementing the same put, open and delete contract, and nothing above the seam changes — we say this in the defence as the honest production answer.

Section E — The Hardest Challenges, Deep Dive (Q51–Q62)

Q51. You claim private file delivery. Prove there is no way to reach a file without authorisation.

Walk the surface with us. There is no static file server and no public bucket: the only paths to bytes are the authenticated download endpoints and the public ticket endpoint. Authenticated downloads pass the middleware chain and an ownership WHERE clause. Public downloads require a ticket that exists only after the atomic share guard passed, is bound to one share, is single-use and dies in five minutes. The bytes themselves are AES-256-GCM ciphertext whose key never leaves the server, so even a hypothetical path leak yields an unreadable file. Every one of those statements is demonstrated in the edge-case script.

Q52. Why is a 403 for someone else's document wrong and a 404 right?

A 403 confirms the resource exists and the caller lacks permission — information itself, useful for enumerating other users' document ids. Because our queries are scoped with owner_id in the WHERE clause, a foreign document simply does not match, and the honest answer is 404 NOT_FOUND, indistinguishable from a typo'd id. This is a standard IDOR defence: do not confirm what you do not have to. The role-escalation demo shows both behaviours side by side — 403 for wrong role on a public route shape, 404 for wrong owner on a resource.

Q53. What stops a revoked share from still working for a few seconds?

Nothing survives, because revocation is checked at request time, not propagated. The revoke endpoint stamps revoked_at on the row, and every subsequent guard UPDATE includes revoked_at IS NULL in its WHERE clause, so the very next open anywhere fails with 409 SHARE_REVOKED. There is no cache of link validity to invalidate, no signed URL still floating around with a clock on it — the link is not a capability, it is a lookup whose validity is recomputed on every use. That design choice is why revocation is instant by construction.

Q54. A share link expires while someone is mid-download. What happens?

Two distinct steps have two distinct guarantees. Metadata and open are guarded: the open call's atomic UPDATE re-checks expiry, so an expired link never yields a ticket. An already-issued download ticket is itself short-lived — five minutes — and single-use, and redemption re-reads the share row's state, so a link revoked after ticket issuance still refuses at redemption. A download already streaming is bounded by the ticket's five-minute worst case, which is the accepted, documented window; tightening it further would trade user experience for a marginal risk.

Q55. How exactly does tamper detection work end to end?

Each version row pins the SHA-256 of its encrypted bytes at upload. On every download the server streams the file through decryption while recomputing the digest and compares the two; a mismatch aborts the response and writes an integrity audit row. The verification feature surfaces the same check to the world: a proof link recomputes the current version's digest and reports tampered instead of valid when it disagrees. The demo overwrites two bytes inside a vault file and shows the verification result flip — evidence that integrity is continuously checkable, not just promised at upload.

Q56. Could two version uploads interleave badly with the download path?

Downloads are safe by snapshot semantics: a download resolves the version row it will stream first, then reads that storage key, so a concurrently committed newer version simply becomes the next download's head. The UNIQUE (document_id, version_number) constraint serialises upload racing as described earlier, and the document row's mirrored head fields are maintained by a trigger rather than application writes, so no interleaving produces an inconsistent triple of current_version, sha256 and storage key. Worst case for a downloader is a slightly older version than the newest one, which is correct behaviour for a snapshot.

Q57. What is your token strategy on the device, and what if the device is stolen?

Tokens live in AsyncStorage on the device and travel only over HTTPS in an Authorization header. A stolen device yields at most the fifteen-minute access token and the refresh token; the thief cannot read source or storage without a locked-phone barrier, and any refresh attempt by the real owner's replacement device after logout revokes the family. Server-side, logout and password change revoke refresh tokens, and suspension kills sessions at the token level on the next call. Refresh-token hashing at rest means even a full database leak does not expose usable refresh tokens.

Q58. How do you handle clock skew in expiry and TOTP?

All server-side expiry comparisons use the database clock — now() in SQL — so link and token expiry are immune to client clocks entirely; clients never assert validity, they only

receive it. JWT verification compares against the server clock, which is the only party that matters. TOTP tolerates the one place user clocks appear: the six-digit code check accepts the current step plus or minus one, the RFC's recommended drift window, and enabling MFA requires a confirmed code so a badly wrong clock is caught at setup rather than at login.

Q59. Why is the verification result its own state machine, not just a boolean?

Because the interesting answer is rarely yes or no. The result distinguishes valid, tampered, expired, revoked and invalid — each produced by a different branch: approved request and matching digest means valid; digest mismatch means tampered; past expiry means expired; deleted vault item means revoked or invalid depending on how the document died. Downstream consumers — the public page, the owner's list, the admin queue — need to show different text and colours per state, and the state machine makes each case a tested unit-test row rather than a conditional in a view.

Q60. What is the most likely production incident, and what does the audit trail give you?

The realistic one is a user reporting an unexpected document change or share. Because document_history is trigger-fed and append-only, support can answer exactly who did what and when with actor, IP and detail payload; because admin actions and refusals land in audit_logs, misuse of authority is visible too. The append-only guarantee means the incident record cannot be edited afterwards, which changes the conversation from whose memory is right to what the row says. We consider that property — tamper-evident operational history — the system's most quietly valuable feature.

Q61. If we asked you to add file previews, what would change?

Only behind the vault seam. Previews are derived plaintext artefacts, so the safe design is a server-side thumbnailer that reads the decrypted stream in-memory, renders a small image, and treats it as its own asset with its own guarded endpoint and no permanent plaintext on disk. The document bytes, their endpoints and the client model stay untouched; the client adds an image tile on the detail screen. We would explicitly forbid generating previews on the client from downloaded originals, because that would move plaintext to devices, which contradicts the delivery story.

Q62. What would you build next with one more week?

Three things, in order. Push notifications, because the inbox plus scheduler already produce the right events and a push transport is additive. Object storage with server-side encryption behind the existing FileVault interface, removing our own disk as an operational concern. And a small performance pass on the admin audit browser — cursor pagination — because OFFSET pagination is the one place where large offsets will eventually hurt. Each is scoped,

each has a named seam, and none would require touching the security core, which is the point of the layering.

Section F — React Native, MVP, Testing and Teamwork (Q63–Q78)

Q63. What does Expo give you, and what does it cost?

We run Expo SDK 58 with React Native 0.88 — the newest SDK generation, bringing the new React Native architecture and iOS 27 support; npx expo-doctor inspects the project like a mechanic and reports 20/20 checks clean. Expo gives a managed toolchain: no Xcode or Android Studio needed to run the app, over-the-air project configuration via app.json, first-class document picking and file system modules, and QR-code preview on real devices — decisive for a student team demoing on borrowed phones. The cost is that exotic native modules require ejecting to a bare workflow, and some native capabilities arrive late to the managed SDK. DocuTrust needs nothing exotic — navigation, storage, document picker, file system are all supported — so the trade was clearly in Expo's favour.

Q64. How is a screen wired together in your MVP? Show the pieces.

Each screen is three roles. The view is a function component that receives state and callbacks from one hook call, usePresenter, which constructs the screen's presenter with its model API and subscribes to its state machine. The presenter is a plain class extending BasePresenter — attach, detach, and named intents like submit() or loadMore() — holding idle, loading, success, error and empty states. The model is the microservice's api.js calling the shared ApiClient. On unmount the hook detaches, so no presenter ever updates an unmounted view.

Q65. Why keep presenters as plain classes instead of hooks inside components?

Because plain classes are trivially unit-testable: instantiate with a mocked API, call methods, assert state transitions — no React renderer, no act() dances, no component tree. That property is what let us write meaningful Jest tests for login, pagination and error flows as the brief's automated-tests requirement demands. It also enforces the discipline that views stay dumb; when logic tries to hide in a component, it shows up as inconsistent loading states across screens, which MVP's shared BasePresenter makes structurally difficult.

Q66. How does the ApiClient's silent refresh work under concurrency?

When any call receives 401 TOKEN_EXPIRED, the client starts a single refresh flight — one module-level promise that all concurrent failures await, so five expiring calls trigger one refresh, not five. The flight posts the refresh token, stores the new pair, resolves, and every waiter retries its original call exactly once; a failed flight clears the session on

TOKEN_REUSE or UNAUTHORIZED. The Jest tests mock fetch and assert precisely this: one refresh network call for N parallel expiring requests, and a retry-once guarantee that prevents loops.

Q67. How do role-based screens work without trusting the client?

The auth store keeps the user object with its role from login and /me, and the navigator mounts the admin stack only for admins and the org screens for managers — that is user experience, not security. The server applies requireRole guards regardless of what the client shows, and every refusal is audited, which the edge-case demo proves by calling an admin endpoint with a member token. So the client hides surfaces for comfort; the server decides authority for truth. Both halves are stated separately in the defence because conflating them is the classic RBAC mistake.

Q68. How do notifications deep-link into screens?

Notification rows carry an entity_type and entity_id, and the client maps the pair through the SCREEN_REGISTRY to a target — an expiry warning opens the document, a share notification opens the share's detail, an admin decision opens the review detail. Tapping marks the row read and navigates. The same registry backs the app's docutrust:// URL scheme for external links such as verification tokens, so one indirection table serves in-app taps, push payloads and web deep links alike.

Q69. What design decisions make the UI feel hand-made rather than generated?

A deliberate non-default design system: an ink-and-parchment palette with a single amber accent instead of the generic blue-purple, serif display headings paired with a quiet sans body, and a letterspaced eyebrow style for labels. Signature components carry personality — rotated dashed stamp badges for statuses, a vertical timeline for history, file-tab cards with a type-coloured edge — and every screen shares one Screen scaffold so the quirk reads as identity, not noise. Empty states are drawn with plain Views instead of stock illustrations, and spacing follows a fixed four-point scale. The goal was a notary's ledger, not a dashboard template.

Q70. Why is your custom Java test runner acceptable instead of JUnit?

Because the brief explicitly forbids JUnit, and the point of the exercise is that a test runner is small. DocuTest is a reflection loop: find public no-argument methods annotated @DocuTest, invoke them, count failures, print a summary, exit non-zero on failure — plus a dozen assertions. It gives us the same workflow as JUnit, red-green suite runs in CI-style scripts, without the dependency, and the runner itself is forty lines a student can explain line by line. The suite it drives is serious: 42 unit tests plus integration tests that boot the real server.

Q71. What do your tests actually assert? Give concrete examples.

Server-side examples: a JWT built by the service verifies and round-trips its claims; an expired token is detected as expired, not merely invalid; a one-bit-changed signature fails; PBKDF2 verify accepts the right password, rejects a wrong one, and produces unique salts; the vault round-trips bytes, fails on a tampered byte and on a wrong key; the router matches /api/documents/:id and prefers static over parameter segments. Client-side: the login presenter transitions loading to success, routes MFA_REQUIRED correctly, and maps errors; the documents presenter paginates and filters; the ApiClient refreshes once for N parallel 401s. Integration and e2e then assert whole flows over real HTTP.

Q72. What is the difference between your integration tests and your e2e suite?

Integration tests run inside DocuTest and boot the actual server against a real database when one is configured, checking migrations, HTTP wiring and database behaviour together; without a database they skip loudly rather than fail, keeping the suite deterministic. The e2e suite is a black box: a Node script that talks to a running server exactly like the mobile app does — register, login, upload multipart, share, open, revoke, verify, admin decisions, notifications — asserting status codes and envelopes, never internals. Integration proves the pieces compose; e2e proves the product works from the outside, which is the distinction an examiner is fishing for.

Q73. How did the six of you divide the work, and why that shape?

Vertical slices: each student owned one domain end to end — server service, screens, tests — so nobody was stuck with only plumbing or only screens and every owner could defend their whole slice. One member acted as architect, owning the router, middleware, JWT and project skeleton that everyone built on; the remainder split auth, documents and vault, sharing and public delivery, database with verification and notifications, and admin with org plus the MVP infrastructure. Pair rotation in week one spread the architecture knowledge, and the feature register's per-module ownership table is the written record of who owns what.

Q74. How did you keep the API contract from drifting during three weeks?

The contract was written first as docs/API_CONTRACTS.md with paths, shapes, envelopes and error codes, and both sides coded against it before endpoints existed; the client developed against the documented shapes using the seeded expectations. Changes were not forbidden but were made by editing the contract first, in a group decision, then both sides converging — the register of 216 features cross-references the same module names, which made drift visible in review. The e2e suite is the enforcer: it runs the contract end to end, so a silent shape change fails a test, not a demo.

Q75. What was the hardest technical disagreement, and how was it resolved?

Whether public download links should be signed URLs over the vault or short-lived tickets redeemed through guarded endpoints. Signed URLs are fewer moving parts, but they cannot be revoked instantly — the signature stays valid until it expires — which contradicts our revocation guarantee. The team chose tickets: an in-memory single-use grant valid five minutes, redeemed through the same guard path as opens. The disagreement is documented in the roadmap's decision log precisely because the losing option was reasonable, and being able to explain why the safer one won is a better defence outcome than unanimity.

Q76. How does the three-week roadmap defend itself against the criticism that it is optimistic?

It is decomposed to the level where optimism can be checked: eleven named migrations, 71 contract paths, 45 registered screens and 232 numbered features each map to a week and an owner, and the two Friday demos are binary gates — a register-login-upload path in week one, the full journey in week two. The risk register names the four realistic failures — Supabase latency in class, multipart bugs discovered late, screen creep, and uneven knowledge — with mitigations already scheduled, like the day-three upload spike and the frozen screen registry. Optimism is cheap; a plan with numbered, testable artefacts is falsifiable, which is what a roadmap should be.

Q77. What is the single weakest part of the system, honestly?

Notification transport. The events, preferences, de-duplication and inbox are real, but delivery is in-app only; there is no push or email, so a user who does not open the app learns about an expiring passport late. It is a product-completeness gap, not an architectural one — the scheduler already emits exactly the payloads a push service would forward — and we name it before the examiner does. Runner-up weaknesses and their production answers are the in-memory ticket map and rate limiter, both needing a shared store when scaled horizontally.

Q78. In one sentence, why should this system be trusted with real documents?

Because every promise it makes is enforced by a mechanism an examiner can watch fail: encryption whose tampering is detectable, delivery whose every path passes a guard, links whose death is checked atomically on use, history that cannot be rewritten, and an audit trail that records the attempts — trust here is not a claim but a set of testable behaviours.

Q79. Your OTP overhaul replaced codes-in-responses with emailed codes. Walk me through the security properties.

Five properties, each checkable. First, at rest: the email_otps table (migration V10) stores only a SHA-256 digest over the purpose, the recipient address and the code itself, so a leaked database row is worthless and cannot be replayed across purposes. Second, in transit: the

code exists only in the recipient's inbox — API responses never carry it; the one exception is the X-Dev-OTP header in MAIL_MODE=dev so demos and the e2e suite work without mail traffic. Third, in time: ten minutes to live, twenty-four for invites, single-use, and burned after five wrong attempts inside one locked transaction so two parallel guesses cannot both spend it. Fourth, in scope: step-up verification issues a separate five-minute, single-purpose JWT (typ=step_up) that destructive endpoints demand in the X-Step-Up-Token header — without it they answer 403 STEP_UP_REQUIRED, and a login challenge token is a different type and is refused there. Fifth, in effect: the old reset-code-in-body and temporary-password-in-body patterns were deleted outright, and the 31-step e2e suite walks every flow including the negative paths.

Per-Student Defence Packs

Each pack maps one member to one of the six defence roles — Infrastructure & Database, Auth & OTP Security, Business Logic, Mobile App, the 8-Pass Security Audit, and Deployment & DevOps — the same roles, pitches and analogies as *DEFENCE_GUIDE.md* at the repository root, so the team tells one story. The two-minute pitch is a script, not a suggestion: deliver it while performing the matching demo action, because a spoken claim plus a live behaviour is the strongest unit of defence. The five personal questions per pack are the ones examiners most often direct at the person who owns that slice; rehearse them aloud until the answers come in three sentences or fewer.

Student 1 — Infrastructure & Database

Role in the team: The skeleton and the truth layer: the pure-Java server platform, the connection pool, and the PostgreSQL schema whose invariants live in the engine.

Owns: Router, middleware chain, JWT service, error envelope, config/boot, migration runner, HikariCP pool, and schema V01 to V11 (15 tables); the Dashboard screen and app shell navigation.

Feature claims: FT-193 to FT-202 and FT-206 to FT-208 (server platform), plus the Dashboard and navigation registry features FT-173, FT-177, FT-178.

Two-minute pitch. *Say this, then demo:*

I own the skeleton everything else stands on, and the truth layer it stands on. The skeleton: a pure-Java HTTP server with a hand-written router mapping our 71 API paths, a middleware chain where every security check happens before business code runs, and a JWT service built on the JDK's HMAC — when we were told no Spring, that was not a limitation for me, it was the job. The truth layer: eleven migrations building fifteen tables whose invariants live in the engine — CHECK constraints that make a sensitive document impossible to activate without

review, UNIQUE constraints that turn upload races into clean conflicts, and triggers that write history automatically and make the audit tables physically append-only. Beneath both sits HikariCP, the connection pool that seats every query instantly. My demo: one request crossing the whole chain live — logged, authenticated, role-checked, rate-limited — and then a manual UPDATE against the history table that the database itself throws out.

Likely personal questions and model answers:

QP1.1. Why should one student own the middleware, rather than sharing it?

Because the middleware chain is where cross-module policy lives, and split ownership would have meant five people editing one file with five opinions. Owning it let me keep the chain small and consistent while module owners focused on services; I still paired with each of them when their endpoints needed a new guard.

QP1.2. Your router matches static segments over parameters. Why does that ordering matter?

If /api/documents/expiring were captured by /api/documents/:id, the word expiring would be treated as an id and the route would 404 or misparse. Matching static segments first, parameters second, is what lets both shapes coexist. The router unit test asserts exactly this precedence.

QP1.3. How do you know your JWT implementation is correct?

Three ways. The unit tests round-trip claims and reject expired, tampered and wrong-secret tokens. The middleware treats expired and invalid differently, which the client depends for refresh. And the e2e suite runs real login-to-refresh flows over HTTP, so the hand-rolled token is exercised end to end, not just in isolation.

QP1.4. If I delete Spring from the equation, what did you actually gain educationally?

The ability to answer how questions that frameworks answer with because it does. How routing matches, when middleware runs, what a 401 versus 403 means, why a connection pool bounds concurrency — each is a file we wrote. That is the difference between using a framework and understanding one.

QP1.5. Why put the sensitive-document rule in a CHECK constraint instead of the service?

Because the service can be bypassed — a migration, a manual SQL statement, a future bug — but the constraint is enforced on every statement by the engine. The rule is one line: a medical or financial document cannot be active unless its review status is submitted.

Encoding it in the schema means the review process has no silent back door, which is exactly what an auditor would want to hear.

Student 2 — Auth & OTP Security

Role in the team: How a stranger becomes a trusted user — and how a stolen password alone still is not enough.

Owns: AuthService, PasswordService (PBKDF2), TokenService (rotation + reuse detection), MfaService, OtpService (hashed one-time codes), MailService (Brevo), StepUpService, and the rate limiter; the eight auth screens.

Feature claims: FT-001 to FT-024 plus the OTP security addendum FT-217 to FT-232.

Two-minute pitch. *Say this, then demo:*

I own how a stranger becomes a trusted user — and how a stolen password alone still is not enough. Registration hashes passwords with PBKDF2 at a hundred and twenty thousand iterations and a per-user salt; login issues a fifteen-minute access JWT plus a seven-day refresh token stored only as a SHA-256 hash; and the refresh endpoint rotates tokens, so presenting an old one after rotation — the signature of a stolen token — nukes the whole token family. This sprint I rebuilt the fragile flows around emailed one-time codes: password reset now emails a six-digit OTP stored only as a hash of purpose, email and code, single-use and dead in ten minutes — the old code-in-the-response anti-pattern is deleted, not deprecated. Invites email a 24-hour code instead of a temporary password, destructive actions answer 403 STEP_UP_REQUIRED until you trade an emailed code for a five-minute single-purpose token, and login can fall back to email MFA through the same OTP engine. In the demo I replay a rotated refresh token on purpose and the audience watches the family die.

Likely personal questions and model answers:

QP2.1. Why store refresh tokens hashed if they are already random?

Because the database should not be able to give a thief a usable token. A hash means a full database leak yields nothing usable directly; the attacker would have to reverse a SHA-256 of a 43-character random string. It costs one hash per refresh and buys immunity to the most damaging dump scenario.

QP2.2. Walk me through exactly what happens when a rotated token is replayed.

The presented token is hashed and looked up; the row is found but already revoked, which by definition only happens if something replayed it. The server answers 409 TOKEN_REUSE and revokes every refresh token belonging to that user, so whatever device holds the stolen

one is locked out with it. The legitimate user simply logs in again — annoying for them, fatal for the attacker.

QP2.3. What exactly changed in the OTP overhaul?

Two anti-patterns died: the password-reset endpoint that returned the reset code in the response, and invites that carried a temporary password in JSON. Their replacements share one engine: six-digit codes from SecureRandom, stored only as sha256 of purpose, target and code, single-use, ten-minute TTL — 24 hours for invites — five wrong attempts burn the code, and one live code per purpose and target. Responses never contain codes in any mode; dev mode puts them in the log and an X-Dev-OTP header, live mode sends real Brevo email. E2E steps 19 to 28 drive every flow including the negative paths.

QP2.4. Why did you return a special 440 status for MFA instead of a normal 401?

The client needs to distinguish two very different situations after a login attempt: wrong credentials, versus correct credentials awaiting a second factor. A distinct machine-readable status makes the MFA screen a routing decision rather than an error message, and it cannot be confused with a failed password. It is a deliberate, documented extension of the standard statuses, used only for that branch.

QP2.5. Could an attacker brute-force the six-digit code?

They get ten attempts per minute per IP from the shared rate limiter, and each attempt must land inside the code's ten-minute life with a five-attempt burn limit, so the search space is practically unreachable. The limiter plus the short window make guessing a losing game, and failed attempts are audited.

Student 3 — Business Logic (Documents, Sharing, Verification)

Role in the team: The bytes and the doors: encrypted storage, versioned documents, and the share engine that faces strangers.

Owns: DocumentService, FileVault (AES-256-GCM), ShareService (atomic guards, passcodes, OTP gating), VerifyService with version pinning; the twelve vault screens and the sharing plus public verify screens.

Feature claims: FT-025 to FT-070, FT-071 to FT-096, and FT-209 to FT-212.

Two-minute pitch. *Say this, then demo:*

I own the bytes and the doors they pass through. On the way in: upload streams your file once, computing its SHA-256 as the bytes flow, encrypts with AES-256-GCM under a key from the environment, and writes ciphertext plus the document row, version one and the first history entry in one transaction. On the way out: a share is a row with a 43-character token, a

permission level, an expiry, an optional view limit and an optional passcode — and its entire policy is one atomic SQL UPDATE that increments the view count only if the link is unrevoked, unexpired and under its limit; zero rows moved means the link is dead, and my service re-reads the row to say exactly why. Downloadable shares add a single-use five-minute ticket, so a leaked URL is worthless twice. My demo is the tamper and the dying link: two flipped bytes in the vault flip a verification from valid to tampered, and a thirty-second share dies in front of the audience.

Likely personal questions and model answers:

QP3.1. Why stream the hash instead of hashing after saving?

Because plaintext should exist in as few places and moments as possible. Streaming lets me compute the digest while transferring to the encryptor in the same pass, so there is no intermediate plaintext file on disk and no second read. It also gives honest file sizes and rejects oversize uploads before we have stored anything.

QP3.2. What exactly does the GCM tag protect against that a plain hash does not?

They protect different axes. The hash tells us what we wrote at upload time; the GCM tag is verified against the key during decryption, so it detects any modification of the stored ciphertext afterwards, even by someone with disk access but not the key. We use both: the tag at decrypt time, the digest for the tamper evidence we can show users and verifiers.

QP3.3. A user uploads two versions at the same moment. Trace it.

Both services read the current max version, both target N plus one, both insert inside transactions; the UNIQUE constraint on document and version number lets the first commit and rejects the second, which we catch and rethrow as a 409 CONFLICT after rolling back — no orphan file, no phantom row. The client then offers a retry, and the loser's staged bytes are deleted. The edge-case script runs this from two terminals simultaneously.

QP3.4. Why one atomic UPDATE instead of check-then-increment in code?

Because check-then-increment is a race: two opens can both pass the check and both increment, overshooting max_views. Making the guard the WHERE clause of the increment means Postgres row locks serialise the two, and the loser matches zero rows — the count can never exceed the limit. One statement is also the audit-friendly unit: rows moved, or not.

QP3.5. You chose tickets over signed URLs for public downloads. Defend that.

A signed URL is valid until it expires; you cannot revoke it early, which breaks our instant-revocation guarantee. A ticket is a server-side grant: single-use, five minutes, bound to one share, redeemed through the same guard path. The cost is one lookup per redemption; the

benefit is that revocation is truly instant for downloads too. We logged this decision in the roadmap because the losing option was reasonable.

Student 4 — Mobile App (React Native 0.88 / Expo SDK 58)

Role in the team: Everything the user touches: 45 screens, nine frontend microservices, the MVP pattern, and the app's own configuration story.

Owns: The Expo client — screens, components, navigation, the microservice folders (api / model / presenter each), BasePresenter and usePresenter, ApiClient with single-flight refresh, mobile/.env with EXPO_PUBLIC_API_URL, and the SDK 58 dependency matrix.

Feature claims: FT-133 to FT-172 and FT-173 to FT-178, plus the mobile side of FT-217 to FT-232.

Two-minute pitch. *Say this, then demo:*

I own everything the user touches. The client is React Native 0.88 on Expo SDK 58 — the newest SDK generation, with the new React Native architecture and iOS 27 support — organized as nine frontend microservices, so six students could work without collisions. The API client handles the boring-but-critical parts silently: it unwraps our response envelope, refreshes an expired access token exactly once with single-flight de-duplication, and surfaces typed error codes instead of raw failures. This sprint I rebuilt the security UX: the login MFA screen offers 'email me a code instead' and finishes the 440 challenge; the password-reset screen takes the emailed six-digit code; and both destructive flows detect the server's 403 STEP_UP_REQUIRED, prompt for the emailed code, trade it for a five-minute step-up token and retry with the X-Step-Up-Token header. Where the app finds the server is configuration, not code: mobile/.env holds EXPO_PUBLIC_API_URL, env.js reads it, and the root .gitignore keeps the real .env out of GitHub. Proof on this machine: expo-doctor reports 20/20 checks clean on SDK 58, the Jest suite runs 74/74, Metro bundles the whole app to Hermes bytecode, and the 31-step E2E drives the same flows against the live API.

Likely personal questions and model answers:

QP4.1. What did the SDK 58 upgrade touch, and what could have broken?

Only configuration. package.json moved to expo ^58.0.0 and every dependency was realigned to the SDK 58 matrix with npx expo install --fix; app.json dropped the splash block that the new schema forbids; .npmrc sets legacy-peer-deps because the jump crosses React 19. No screen, presenter or API code changed. What could have broken — native module versions and the Jest preset — is exactly what the verification covers: expo-doctor 20/20, Jest 74/74 on the new jest-expo, a clean Metro bundle, and the full E2E against the live server.

QP4.2. Why is the screen registry more than a folder of imports?

Because it is the single indirection point that navigation, deep links and notification taps share: a name maps to a component, its owning module and its title, and nothing navigates by constructing paths ad hoc. That gave us role-based mounting — the admin stack exists only for admins — and a countable inventory: forty-five entries is not a claim, it is a file. When a screen moves or a module changes hands, the registry is the one place that records it.

QP4.3. What does BasePresenter standardise, concretely?

Five states — idle, loading, success, error, empty — plus attach and detach and a shared error mapper. Because every screen passes through it, every screen shows the same skeletons, the same empty states and the same toast behaviour, and the Jest tests drive presenters with mocked APIs without any React rendering. The pattern is why forty-five screens feel like one app instead of forty-five prototypes.

QP4.4. Your Jest tests test presenters, not components. Why is that the right cut?

Because the bugs that matter live in decisions, not rendering: does login route MFA correctly, does pagination accumulate, does the client refresh once for five parallel 401s. Presenter tests run in milliseconds with mocked APIs and no DOM, so we could actually write and maintain them in the time we had. Component snapshots would have looked impressive and caught nothing we cared about.

QP4.5. Is client-side token storage in AsyncStorage secure enough?

It is the platform-standard trade-off for this kind of app: AsyncStorage is sandboxed per app, so another app cannot read it, and the real threats are device theft and device-level compromise. We mitigate what we can from the server side — fifteen-minute access tokens, rotation with reuse detection, revocation on logout and password change — and a production roadmap item would be encrypted storage or biometric-gated keychain use.

Student 5 — The 8-Pass Security Audit

Role in the team: Attacking our own system before an examiner could: eight thematic passes, finding IDs that live in the code comments, and the regression proof that fixes broke nothing.

Owns: The eight audit passes and their finding IDs (A-1 to H-3), the three hardening batches that fixed them, and the regression suite — 42 unit tests plus 31 E2E steps green after every batch.

Feature claims: Audit findings and regression proof; every ID cited in code comments is demonstrable.

Two-minute pitch. *Say this, then demo:*

My job was to attack our own system before an examiner did. I ran eight thematic passes over the original code — access control, crypto and vault, concurrency, configuration, input validation, information leakage, error handling, and data integrity — and logged every finding with an ID and a severity. The fixes shipped in three tested batches, and the IDs stuck: B-1 was refresh-reuse detection that used to fail open — the revocation was rolled back by the very exception that reported it, so a stolen token stayed usable; C-1 was nested transactions holding two pool connections at once, which could commit an audit row even when the outer transaction rolled back — a phantom audit; E-3 made the server refuse to boot on weak JWT secrets; D-1 pinned the vault key to exactly 32 bytes. Every fix landed with a test, and the full suite — 42 unit, 31 end-to-end — is green on the final build. The audit trail is in the code: search any ID letter and you will find the comment explaining what broke and why the fix is shaped the way it is.

Likely personal questions and model answers:

QP5.1. What was your most serious finding?

B-1, the refresh-token reuse detection that rolled back its own revocation. It is the nastiest class of bug — the security control exists in the code, there is even a test path for it, but the transaction boundary made it fail open exactly when it mattered. The fix separates 'revoke the family' into its own committed transaction before the 409 is raised, with the audit event written best-effort afterwards so an audit outage can never undo a revocation. You can read the whole story in the comments around TokenService.rotate.

QP5.2. Explain the phantom audit bug — why is it dangerous?

C-1. The old code opened a second connection inside an outer transaction to write the audit row, and the two committed independently. So a business operation could roll back — the thing never happened — while its audit row survived: the trail then claims events that never occurred, which is worse than a missing row in a system whose promise is tamper-evident history. The fix is transaction hygiene: one connection per transaction through the shared runner, and post-commit side effects written best-effort, so the trail can only lag, never lie. The regression test forces a failure and counts audit rows.

QP5.3. How did you prioritize eight passes' worth of findings?

By exploitability times blast radius. Anything that failed open, leaked a credential, or crossed an ownership boundary was batch 1 — 'someone can get in or see someone else's data'. Integrity and tamper-evidence was batch 2. Concurrency and robustness cleanups were batch 3. And documentation-level risks got logged with a named fix path rather than pretend-fixes. Nothing was closed because we felt like it — each closure points at a test.

QP5.4. Did the audit ever produce a false alarm?

Yes, and that is worth saying out loud. The OTP overhaul initially looked like a regression source when the E2E suite hit 429s — but the limiter was doing its job: a 31-step suite from one IP legitimately exhausts a 10-per-minute auth bucket, and a real user doing password-reset, MFA and step-up in a minute would too. The honest fix was a scope-aware budget — 20 a minute for auth, 30 for public share endpoints — not disabling the limiter. Knowing the difference between 'the system is broken' and 'the system is protecting itself' is the core audit skill.

QP5.5. Give one finding that was not a classic security hole.

H-1: the default document download resolved to the wrong version when the current-version pointer and the mirror disagreed, so a user could fetch bytes other than the version they were shown. The fix pins download resolution to one definition of current. H-3 was an orgName join that silently dropped organisations with unusual casing, and F-4 pinned the client's document versions so a stale screen cannot overwrite a newer one. None of these leak data on their own — all of them break a promise the UI makes, which is why they were worth a pass of their own.

Student 6 — Deployment & DevOps

Role in the team: Getting it running — anywhere, in one command — and keeping it observable, repeatable and secret-clean.

Owns: Maven pom.xml and the dependency fetcher (fetch-libs.sh), compile-check.sh (javac with an ECJ fallback), run-dev.sh and run-tests.sh, .env configuration with fail-fast validation, the root .gitignore and the mobile env files, the demo seeder, the ExpiryScheduler, metrics and health endpoints, and optional Sentry Spotlight tracing.

Feature claims: Infrastructure libraries (HikariCP, Bucket4j, Caffeine, Dropwizard Metrics, Sentry) plus the operations surface: boot, health, metrics, scheduler, seed.

Two-minute pitch. *Say this, then demo:*

I own getting it running — anywhere, in one command. The server is a single Java 21 process: run-dev.sh fetches the exact library jars from Maven Central, compiles if needed — with javac, or ECJ when only a JRE is present — then boots with a fail-fast config check: a weak JWT secret or a missing vault key stops startup with a human explanation instead of a runtime surprise. Configuration is environment variables documented in .env.example, including the Brevo mail credentials and the MAIL_MODE switch — dev mode routes OTP codes to the log and an X-Dev-OTP header for demos, live mode sends real email. Secrets never reach the repository: a comprehensive root .gitignore blocks .env, node_modules, build output and IDE folders, and git check-ignore proves any pattern live. Operations got real telemetry: HikariCP pool stats and Dropwizard request metrics surface on /api/health,

and Sentry Spotlight tracing is one optional flag. Proof: from a clean clone, bash server/run-dev.sh plus the mobile E2E — 31 green steps — and the boot log prints migrations=11, routes=71.

Likely personal questions and model answers:

QP6.1. How would you deploy this to real users?

The server is a single jar with no framework: any JVM 21 host or container works. I would run it behind a reverse proxy for TLS (the API already sets CORS per config), point DB_URL at managed PostgreSQL — we already support the Supabase pooler form — set MAIL_MODE=live with the Brevo key, VAULT_KEY from a secret store, SEED_DEMO=false (the server itself refuses demo seeds on non-localhost databases), and let the migration runner bring the schema up on first boot. Health checks already exist at /api/health.

QP6.2. What stops a developer from leaking production secrets into the repo?

Three fences. First, secrets only exist in .env files, and a comprehensive .gitignore sits at the project root blocking .env, target/, node_modules/, dist/, build/, IDE folders (.idea/, .vscode/, *.iml) and OS files (.DS_Store, Thumbs.db) from ever being committed — you can verify any pattern live with git check-ignore. Second, .env.example documents every variable with placeholders that the server actively refuses — the JWT placeholder is on a blocklist (E-3) — so nobody can run with example secrets by accident. Third, the committed template files carry no real values: mobile/.env.example ships YOUR_PC_IP_ADDRESS as its placeholder.

QP6.3. If the server crashes at 3am, how do you know what happened?

Three layers. Every request logs one line — method, path, status, duration. Every security-relevant event lands in the append-only audit_logs table with actor, IP and JSON detail. And /api/health plus the Dropwizard counters on /api/meta show whether the database and the request path are healthy. Optional Sentry Spotlight tracing adds the distributed-timeline view when it is switched on. The logs answer 'what happened'; the audit trail answers 'who did it'; the metrics answer 'is it still happening'.

QP6.4. The machine running the demo has no Maven and no JDK. Now what?

That is exactly the path compile-check.sh takes. fetch-libs.sh downloads the eight pinned runtime jars straight from Maven Central — HikariCP, Bucket4j, Caffeine, Metrics, Sentry, the Postgres driver — and the compile falls back to the Eclipse compiler as a single jar when javac is absent, so a plain JRE is enough to build and run. We proved it this sprint: a cold build from the submission zip on a JDK-less machine reached a green compile and 42

passing unit tests. Same code, same jars, no network surprises because the versions are pinned.

QP6.5. How does the scheduler avoid notifying twice about the same expiry?

One notification per document, per type, per calendar day: before inserting, the scheduler checks for today's existing row with the same entity and type. A document sitting in the expiring state for two weeks produces fourteen reminders, not hundreds. The same sixty-second tick retires expired verification requests so their public links stop answering valid.

Demo-day survival tips. Arrive with the server already running and the seed data fresh, because first boots are for the preparing, not the performing. Keep three terminal windows ready: one tailing the server log, one with the demo curl scripts from docs/EDGE_CASES.md, and one with the SQL editor open for the append-only exception demo. When a question stalls you, attach it to a demonstration — show the row, the log line or the test — and if you genuinely do not know, say what you would look at first; a precise unknown beats a vague guess every time.

Appendix — The Eight Core Concepts, Explained for Absolute Beginners

This chapter assumes the reader has never written a line of code. Each of the eight sections explains one core concept with an everyday analogy, assigns it to the student who should rehearse it hardest, and ends with a one-sentence version you can say when the examiner asks for the short answer. Every folder name, number and library mentioned here is real — open the folder during the defence and point at it.

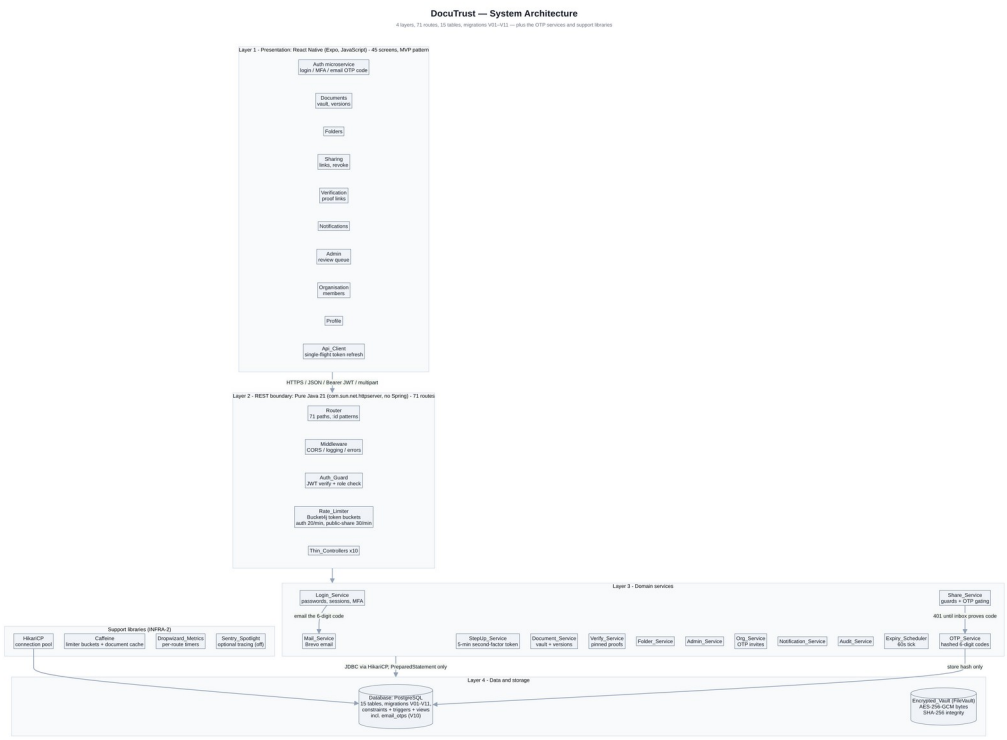


Figure A. The whole system in one picture: the React Native client, the pure-Java server with its 71 routes, and the PostgreSQL pantry beneath.

1. React Hooks — the sticky notes that remember things (rehearse: Student 4)

A phone app screen is redrawn hundreds of times a second — every tap, every new piece of data. But a drawing has no memory: wipe it and start again, and nothing is left over. A **hook** is a sticky note that React — the drawing library — keeps beside the drawing. The screen is wiped and redrawn all day long, but the sticky notes stay on the desk, remembering things between redraws. The word hook literally means that your screen component hooks into React's memory. Three sticky notes matter in our app.

useState — the sticky note with a magic pen. You write a value on the note, for example "unread notifications: 3", and the screen copies whatever is on the note. When a new notification arrives, you call the pen that came with the note; the note now reads 4, and React automatically redraws the screen. You never redraw anything by hand — changing the note is the whole job. Every counter, form field, loading flag and error message in our 45 screens lives on such a note.

useEffect — the "after you finish drawing, run this errand" slip. Drawing a screen should stay fast and dumb. The errands — fetching the document list from the server, starting a countdown, subscribing to notifications — go on the slip, and React runs each errand after the drawing is done. The slip also has a cleanup line: when the screen closes, the timer is

stopped and the subscription cancelled, so nothing keeps running in the background like a tap left on.

usePresenter — our own custom sticky note, the one an examiner will ask about. DocuTrust does not let screens talk to the server directly; each screen gets a **presenter** — a plain-JavaScript "waiter" holding the screen's brain: what to fetch, what counts as a valid form, what to show while loading, on error, on empty. `usePresenter` is the three-line glue that hires the waiter when the screen opens, passes the waiter's status onto a sticky note so the screen redraws as it moves idle → loading → success → error, and fires the waiter when the screen closes so no presenter haunts a closed screen. One hook, and 45 screens all behave the same way.

One-sentence version: a hook is a sticky note that remembers things between redraws; useState remembers a value, useEffect remembers an errand to run after drawing, and our usePresenter remembers to hire — and later fire — the screen's waiter.

2. The Overall Architecture — the customer, the waiter, the kitchen (rehearse: Student 1)

DocuTrust is a restaurant with three rooms. The **customer** is the mobile app on your phone, built with React Native and Expo. It is good-looking and polite, but it has no kitchen: it cannot store your documents, check passwords or decide who may see what. It only orders.

The **waiter** is the backend server, a single Java program. Every wish from the customer — "show my documents", "share file X until Friday", "email me a login code" — is spoken to the waiter, never shouted into the kitchen. The waiter does four things, always in order: checks you are who you claim to be (a digital ID card called a JWT), checks you are allowed to order that (role and ownership rules), walks the order into the kitchen, and carries the plated result back on a standardized tray (a JSON envelope, explained in section 8). Our restaurant has exactly 71 dishes on the menu — the 71 registered API routes, a number printed when the server boots, so it is checkable rather than claimed.

The **kitchen and pantry** are the services and the PostgreSQL database. The chefs — the service code such as DocumentService, ShareService and OtpService — do the real cooking: encrypting files, minting share links, hashing one-time codes. The pantry is the database: fifteen filing drawers (tables) that remember every document, user, share link and audit entry, with a strict head clerk who refuses orders that break the house rules even when the chef made a mistake. Why three rooms? So each can be fixed alone: this sprint the entire OTP flow was rebuilt inside the kitchen, and the customer barely noticed — only the ordering script changed.

One-sentence version: the app is the customer, the server is the waiter with 71 things on the menu, the database is the pantry — and no room ever walks into another room uninvited.

3. The Frontend Folders — it's like a smartphone (rehearse: Student 4)

Open mobile/src/ and you see seven folders. The easiest way to explain them: the app is a smartphone, and each folder is a part of the phone.

`screens/` are the different **apps you open** — forty-five of them: a Login app, a Documents app, a Trash app, a Share app. Each screen is one full-page view, and they are deliberately dumb: they draw what they are told and forward taps, like an app that only shows what its data says. `components/` are the **buttons on the screen** — small reusable pieces (a document card, a password input, a toast pop-up, a loading skeleton) drawn once and used on many screens, which is why the whole phone feels consistent.

`microservices/` are the **invisible background processes that fetch your data** — nine folders (auth, documents, folders, sharing, verification, notifications, org, profile, admin), one per feature area. Each holds that domain's api files (how to ask the waiter), model files (what the data looks like) and presenter files (the screen's brain). The user never sees these; like background processes they do the fetching and deciding. Why separated? So six students could work at once without stepping on each other — the sharing student rebuilt her whole OTP-gating flow and never touched the documents folder. The remaining folders are small: `navigation/` is the home-screen gesture (which app opens when you tap what), `mvp/` is the house rulebook every screen's waiter follows, and `core/` is the phone's settings — the server address and the design tokens.

One-sentence version: screens are the apps you open, components are the buttons on them, microservices are the invisible background processes fetching the data — and navigation is the home gesture that moves between apps.

4. The Backend Folders — a well-organized restaurant (rehearse: Student 1)

Open server/src/main/java/com/docutrust/ and you see the restaurant's staff chart. The `controller` folder is the **waiters taking orders** — ten of them (AuthController, DocumentController, ShareController and so on), one per section of the menu. A waiter's job is narrow on purpose: receive the order, check the ID, hand it to the kitchen, carry the plate back. Waiters never cook.

The `service` folder is the **chefs cooking** — the real logic: hashing one-time codes, encrypting files, enforcing share rules. Chefs never talk to the customer directly; that is the waiter's job. This split is why the team could swap the recipe for password resets — from

"send a temporary password" to "send a one-time code" — without changing a single waiter. The `db` folder is the **pantry and the head clerk**: Database (the walk-in cooler connection), Migrator (the eleven numbered instruction sheets that build the pantry from scratch), Rows and Tx (the rules for reading a shelf and for all-or-nothing orders). The `vault` folder is the restaurant safe — files sit there encrypted with AES-256-GCM, a standard strong enough for banks.

The `auth` folder is the **ID checkers at the door**: PasswordService (checks your password without ever knowing it — it only compares scrambled fingerprints), TokenService (prints and re-prints your session wristband), MfaService (the second-proof checker) and RateLimiter (the bouncer who says "too many tries, wait a minute"). The `http` folder is the **uniforms and order pads** — Router (the 71-item menu index), Middleware (the security chain every order passes), Ctx (the standard order-slip format). Around them: config is the manager's opening checklist (it refuses to open with weak keys), infra is the office dashboard showing pool and traffic statistics, model holds the standard plate shapes, and util is the toolbox. Why separated? A new chef can be hired without retraining the waiters — and during the OTP overhaul the team rewrote chefs (OtpService, MailService) while the waiters and the pantry never changed.

One-sentence version: controllers are the waiters taking orders, services are the chefs cooking, db is the pantry, auth checks IDs at the door — and no waiter ever walks into the pantry himself.

5. The Database — a highly organized filing cabinet (rehearse: Student 1)

A **relational database** is a filing cabinet with strict rules, and PostgreSQL — the one we chose — is the strictest, most trusted cabinet on the market. The cabinet has fifteen drawers (**tables**): users, documents, document_versions, shares, email_otps, audit_logs, and so on. (Peek with a database tool and you will see a sixteenth — schema_migrations — but that is not one of ours: it is the moving truck’s own clipboard, where the migration runner writes down which numbered instruction sheets it already delivered.) Inside each drawer are **rows** — index cards, one per user, per document, per share link. Each card has the same printed **columns**: a user card always has name, email, role, created-at. You never find a hand-scribbled card with a different shape — the drawer's form is fixed when the drawer is built.

A **foreign key** is a cross-reference written on the card. A document card says "owner: see card 42 in the USERS drawer." The clerk refuses to file a document whose owner card does not exist, and — by our setting — refuses to destroy a user card while documents still point at it. That is what "relational" means: the cabinet itself guarantees the references make sense, so a typo or a bug cannot create an orphan.

Why PostgreSQL, in plain words? Four reasons. First, it **enforces rules**: a CHECK constraint is a house rule the clerk applies even to the boss — a medical document physically cannot become "active" without review, no matter what buggy code asks for. Second, it **keeps honest history**: triggers make the audit cards append-only — they cannot be edited or torn out, only added. Third, it is **free and battle-tested for over twenty-five years**, running everywhere — including Supabase, a free cloud landlord for it, so our project costs nothing to host. Fourth, it speaks a standard language, SQL, that all of us could write. The cabinet was assembled from **eleven numbered instruction sheets (migrations)**: V01 builds the users drawer, V10 adds the email-OTP drawer, V11 adds three fields to shares — so any team member can rebuild the identical cabinet from nothing, in order, in minutes.

One-sentence version: tables are drawers, rows are index cards, foreign keys are cross-references the clerk refuses to break — and PostgreSQL is the strict, free, 25-year-old cabinet that enforces our house rules even against bugs.

6. The Server — a 24/7 receptionist answering phones (rehearse: Student 6)

What is a web server? It is a program that never sleeps, sitting by a phone that never stops ringing. Ours is one single Java process, and its day looks like this. The phone rings — a request arrives from the app. The receptionist picks up **within milliseconds, always**, whether it is one call or fifty at once. It asks "who is calling?" and checks the caller's ID card (the JWT): no card, or a fake card, gets a polite refusal — status 401. It looks up the requested department in its directory of **71 extensions** (the router) — and anything unknown gets a clean "no such extension" (404), never a crash. Then it walks the request to that department and carries the answer back, always on the same standardized tray (the JSON envelope), with a rubber stamp saying how it went.

Three more duties make ours a good receptionist. It **writes down every call** — one log line per request (what was asked, when, how long it took), and every security-relevant event into an append-only audit book that even the boss cannot tear pages from. It **limits repeated callers** — someone asking for a password reset eleven times in a minute is told to wait (rate limiting), which is how the system shrugs off people who try to guess codes. And it **refuses to open the office with broken locks** — at start-up it checks its own keys and configuration, and if the signing key is weak it refuses to start and prints exactly what to fix. That is called fail-fast: a two-minute fix on the ground beats a mystery crash in the air.

One-sentence version: a web server is a 24/7 receptionist: answers every call instantly, checks IDs, forwards to one of 71 departments, writes down everything, and never hangs up rudely.

7. The Tech Stack — what we chose and why (rehearse: Student 6)

Every technology in the project earns its place in one of three ways: the brief told us to, it is the boring trusted option, or it is the standard library for a problem too dangerous to hand-roll. The table below is the whole list — learn the "why" column and the examiner's question is already answered.

Java 21 is the language of the server — one recipe book that runs on any machine with a Java reader installed. The brief forbids frameworks, and Java is a twenty-five-year-old workhorse: strict (mistakes are caught before running), fast, and familiar to all of us; version 21 is a long-term-support release, the sturdy supported-for-years edition. **PostgreSQL, hosted free on Supabase**, is the filing cabinet of section 5 — strictest rule enforcement, free hosting, decades of trust. **React Native 0.88 with Expo SDK 58** is the phone-app builder: you write the app once in JavaScript and it runs on both iPhone and Android, like a translator fluent in both — and Expo is the training wheels: scan a QR code and the app runs on a real phone immediately, no app store needed for demos. The app even has its own mechanic: `npx expo-doctor` inspects the dependency list and currently reports 20/20 checks clean, and `npx expo install --fix` keeps every library on the version the SDK certified. Where the app finds the server is configuration, not code: `mobile/.env` holds EXPO_PUBLIC_API_URL and `src/core/config/env.js` reads it, while a root `.gitignore` keeps the real .env, node_modules and build output out of GitHub.

Brevo is the postman — an email-delivery service with a simple API. One-time codes must reach real inboxes; Brevo has a free tier and a two-line sending call, and our MAIL_MODE switch lets us rehearse the whole OTP flow without emailing anyone. **HikariCP** is the restaurant host for the database: it keeps a fixed set of tables (connections) ready and seats each query instantly — hand-rolling that seating is a classic source of 3am crashes. **Bucket4j** is the ration-book stamp that counts requests per minute per user, with budgets tuned per endpoint scope (20 a minute for auth, 30 for public share pages). **Caffeine** is the noticeboard by the kettle — the few hot facts everyone re-reads, kept in memory with size and time limits so the board can never go stale. **Dropwizard Metrics and Sentry Spotlight** are the office dashboard and the CCTV timeline: /api/health shows pool and request statistics live, and Spotlight (optional, one flag) draws the full timeline of a request across the system.

One-sentence version: every choice is either "the brief told us" (pure Java, no framework), "the boring, trusted option" (PostgreSQL, Java 21 LTS), or "the standard library for a problem too dangerous to hand-roll" (HikariCP, Bucket4j) — and we can name the danger for each.

8. The Data Consumption API — the standardized order form (rehearse: Student 3)

How does the app actually talk to the waiter? Two conventions make it work, and both are agreed paperwork. **HTTP methods are the verbs on the order pad**. Every request says what kind of wish it is. **GET** means "just let me look" — opening the documents list changes nothing. **POST** means "here is something new" — uploading a document, registering, requesting an OTP code. **PUT** means "replace this whole thing" — overwriting the notification settings. **PATCH** means "amend one field" — renaming a document. **DELETE** means "take this away" — moving a document to trash. The same address can behave differently by verb: GET /api/documents lists, POST /api/documents uploads, DELETE /api/documents/12 trashes document 12. Reading is safe; writing is checked twice — ID first, then permission.

JSON is the standardized order form — a way of writing data that both the app and the server read: labelled boxes in fixed positions, like {"title": "Passport", "status": "active"}. The phone speaks Android and the server speaks Java, but neither cares: they only swap forms. Every answer arrives in the **same envelope**: ok true with the data inside on success; ok false with an error code and a friendly message on failure — so the app opens every envelope the same way, and a screen that hits an error always finds a code it can translate into plain words.

The rubber stamp on the envelope is the **status code**, three numbers with fixed meanings worth memorising: 200 "here you go"; 400 "your form was filled wrong"; 401 "ID card missing or fake"; 403 "real ID, but not allowed" — or, with STEP_UP_REQUIRED, "allowed, but prove a second time first"; 404 "no such thing — and we will not say whether it exists for someone else"; 409 "conflict — the link is expired, revoked or out of views, or that token was reused"; 429 "too many requests, wait a minute"; 440 — our own stamp — "second factor needed, check your email"; 500 "our mistake, sorry — the details are in the log, not in your face."

One-sentence version: the frontend hands the backend standardized order forms — five verbs on the pad, JSON as the form's printed boxes, one envelope for every answer, and a rubber stamp (the status code) saying exactly how it went.

These eight explainers are also in the repository's *DEFENCE_GUIDE.md* (Part 2), next to the six per-student defence packs — the two documents share the same analogies on purpose, so the team tells one story.